

ADR-001 — Décomposition en microservices avec Bounded Contexts DDD

Champ	Valeur
Statut	Accepté
Date	2026-01-15
Auteurs	Équipe CanBankX

Contexte

CanBankX est une plateforme bancaire numérique qui doit gérer :

- L'inscription et l'authentification des clients (KYC, MFA)
- La gestion des comptes bancaires (CHECKING, SAVINGS)
- Le traitement des paiements (DEBIT, CREDIT, TRANSFER)

Ces trois domaines ont des cycles de changement distincts, des modèles de données qui ne se recoupent pas, et des exigences de charge très différentes (les paiements sont beaucoup plus sollicités que l'inscription).

Les options considérées étaient :

1. **Monolithe modulaire** — un seul déployable Spring Boot
2. **Microservices par bounded context** — un service par domaine DDD
3. **Microservices fins** — un service par entité (trop granulaire)

Décision

Nous adoptons l'option 2 : **trois microservices alignés sur les bounded contexts DDD** :

Service	Bounded Context	Entité racine	Port
<code>identity-service</code>	Identité & Authentification	<code>Client</code>	8081
<code>account-service</code>	Gestion des comptes	<code>Account</code>	8082
<code>payment-service</code>	Paiements & Transactions	<code>BankTransaction</code>	8083

Un module `common` partagé contient uniquement les contrats transverses (gestion d'erreurs normalisée, DTOs partagés).

Conséquences

Positives

- **Isolation des domaines** : chaque service possède son propre schéma MySQL (`db_identity`, `db_account`, `db_payment`), empêchant le couplage de données.
- **Déploiement indépendant** : les services peuvent être mis à jour séparément sans affecter les autres.
- **Scalabilité ciblée** : `payment-service` (le plus sollicité) peut être répliqué indépendamment des autres.
- **Testabilité** : les tests de charge k6 ont confirmé que `identity-service` tient à 200 VUs (p95 = 25 ms) même quand `payment-service` est saturé.

Négatives / Risques

- **Appels inter-services synchrones** : `payment-service` appelle `account-service` via HTTP REST pour les opérations de débit/crédit. Un échec de `account-service` bloque le traitement du paiement.
- **Consistance éventuelle** : il n'y a pas de transaction distribuée. En cas de panne partielle, le mécanisme de compensation (rollback du débit si le crédit échoue) est implémenté manuellement dans `PaymentService`.
- **Complexité opérationnelle** : nécessite un API Gateway (→ ADR-003), orchestration Docker Compose, et une observabilité distribuée (→ Prometheus multi-job).